hacker-dashboard — Product Requirements Document

문서 버전: v1.0 · **작성일**: 2026-04-22 · **상태**: MVP 완성 (공모전 제출 직전) **리포지토리**: hacker-dashboard · **브랜치**: `feat/qa-harness-20260422-111542-nl-copilot-sprint-05`

---

1. Executive Summary

hacker-dashboard는 임의 투자 데이터 스키마(CSV)를 업로드하면 **자동으로 분석 뷰를 생성**하는 범용 금융 대시보드입니다. **Meta Router 에이전트**가 자산군(주식·코인·환율·매크로·포트폴리오)을 동적 판별하고, 자산군별 **Sub-Analyzer**가 LangGraph 서브그래프로 분석을 수행하며, **3단 품질 게이트**(스키마·도메인·자기비판)를 통해 LLM 환각을 차단합니다.

핵심 차별점은 "결정적 수치 + LLM 해석 분리" 설계로, 리밸런싱·집중도·수익률 등 수치는 순수함수가 계산하고 LLM은 자연어 설명만 담당하므로 숫자 환각이 원천 차단됩니다. 4주 공모전 일정 내에 자동 반복 하네스(Planner → Generator → Evaluator 루프)로 품질을 검증했으며, 6개 스프린트 평균 9.2/10 점을 달성했습니다.

2. 배경과 문제 정의

2.1 문제

- 개인 투자자·리서처는 증권사·거래소마다 제각각인 CSV 포맷을 직접 정리해야 분석에 쓸 수 있음
- 시중 대시보드는 **사전에 정의된 스키마**만 받는 경직된 구조
- LLM 기반 분석 툴은 수치 환각 위험이 크고 근거 추적이 어려움

2.2 기회

- Claude Sonnet/Opus 4.x 의 도구 호출 품질이 상용 수준으로 향상 → 도메인 분석 파이프라인에 안전하게 투입 가능
- LangGraph 서브그래프 패턴으로 **자산군별 전문 체인**을 모듈화할 수 있음
- Next.js 15 App Router + SSE 스트리밍으로 "5초 내 첫 인사이트" 체감 가능

2.3 타겟 사용자

페르소나	니즈
개인 투자자	여러 거래소 CSV를 한 화면에 통합, 분산/집중도 체크
리서처 / 애널리스트	임의 종목군에 대해 즉시 시계열·상관 분석
심사위원 (공모전)	5초 내 데모 임팩트 + 기술 깊이 양립 확인

3. 제품 목표

구분	목표	측정 지표
North Star	CSV 업로드 → 분석 뷰 렌더까지 5초 이내	p95 렌더 시간
기술 깊이	Router/Analyzer 다중 에이전트 구조 + 3단 게이트	ADR 수, 게이트 통과율
데모 임팩트	공모전 8분 시연 시나리오 완수	리허설 스톱워치
완성도	Lighthouse 90+, 다크모드, 에러/빈 상태 처리	4페이지 Lighthouse 실측
코드 품질	린터/타입체커 0 경고, 테스트 그린	mypy·ruff·tsc·pytest·vitest

4. 범위

4.1 포함 (In Scope)

- CSV 업로드 → Router 자산군 판별 → Analyzer 분석 → 카드형 UI 렌더
- 포트폴리오 보유 자산 조회·리밸런싱 제안 (순수함수 계산 + LLM 해석)
- 워치리스트 CRUD, 종목 상세 페이지 (TradingView Lightweight Charts)
- 자연어 코파일럿 (`/copilot/plan` , `/copilot/query` SSE 스트리밍)
- 세션 메모리 기반 멀티턴 대화 + follow-up 라우팅
- 3단 품질 게이트 (스키마·도메인·자기비판)
- 데모용 고정 샘플 데이터 (실거래 API 키 없이 동작)

4.2 제외 (Out of Scope, v2)

- 사용자 인증/권한 (공모전 범위 외)
- 실시간 시세 API 통합 (현재 avg_cost 폴백; Upbit/Yahoo 연동은 v2)
- FX 자산군 리밸런싱 (0 고정; 주식·코인·현금만 리밸런싱)
- 포트폴리오 외 자산군 신규 매수 추천
- 모바일 네이티브 앱

5. 핵심 아키텍처

5.1 계층 구조

```
[ Next.js 15 (Vercel) ]
        ⇕  OpenAPI / SSE
[ FastAPI + LangGraph (Fly.io) ]
        ⇕
[ Meta Router ] ──► [ Sub-Analyzer × 5 ] ──► [ 3단 품질 게이트 ] ──► Response
```

```
                    └── [ Postgres (Neon) · Redis Cache ]
```

5.2 Meta Router (`backend/app/agents/router.py`)

- **결정적 heuristic 우선**: 정규식 패턴(KRW-BTC, 005930.KS, USDKRW=X) + CSV 컬럼 시그니처로 자산군 판별
- **Router reason 마커**: heuristic 결정 시 `(heuristic)` 접미사 부착 → 데모 UI에서 LLM 호출 여부 즉시 식별
- **신뢰도 매트릭스**: hint 우선(1.00) → holdings 감지(0.98) → 티커 패턴(0.95) → LLM 폴백(0.70)

5.3 Sub-Analyzers (`backend/app/agents/analyzers/`)

모듈	역할
`base.py`	공통 지표 (SMA, RSI, 수익률, 집중도)
`stock.py` / `crypto.py` / `fx.py` / `macro.py`	자산군별 시계열 분석
`portfolio.py`	보유 자산 통합 분석
`rebalance.py`	리밸런싱 제안 (결정적 액션 + LLM 해석)
`comparison.py` / `simulator.py` / `news_rag.py`	Copilot 확장 에이전트

5.4 3단 품질 게이트 (`backend/app/agents/gates/`)

1. **스키마 게이트**: Pydantic v2 출력 파싱 → 실패 시 1회 재시도 + 수정 지시
2. **도메인 게이트**: 가격 > 0, 시간 단조 증가, 자산군 일관성 등 sanity check
3. **자기비판 게이트**: 별도 LLM 호출로 "근거가 데이터에 있는가?" 검증

각 통과 여부를 응답 메타 `gates: {schema: "pass", domain: "pass", critique: "pass"}` 로 노출.

5.5 Copilot 오케스트레이터 (Sprint-04~05)

- `POST /copilot/plan` — 자연어 질의 → multi-step `CopilotPlan` (JSON 단일 응답)
- `POST /copilot/query` — SSE 스트리밍으로 단계별 카드 progressive 렌더
- `GET/DELETE /copilot/session/{session_id}` — 세션 메모리 (UUID 강제 + parameterized SQL)
- **CopilotCard discriminated union** — 6 variant: `text`, `chart`, `scorecard`, `citation`, `comparison_table`, `simulator_result`

6. 기술 스택

계층	기술
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind, shadcn/ui, TradingView Lightweight Charts

계층	기술
Backend	FastAPI, LangGraph (서브그래프), Pydantic v2, Python 3.12
LLM	Claude Sonnet 4.6 (기본), Claude Opus 4.7 (고난도), 프롬프트 캐시
Database	Neon Postgres 16 (서버리스, branch-per-PR), Redis 7 (30min TTL)
Infra	Docker Compose (로컬), Vercel (FE), Fly.io/Render (BE), GitHub Actions CI
품질	Ruff, Black, Mypy (strict), ESLint, Prettier, tsc, Pytest, Vitest, Playwright

7. API 사양 (OpenAPI 기준)

메서드	경로	용도
GET	`/health`	헬스체크
GET	`/market/symbols`	지원 심볼 목록
GET	`/market/symbols/search?q=`	4개 소스 병렬 검색 (asyncio.gather)
POST	`/portfolio/upload`	CSV 업로드 → Router + Analyzer 파이프라인
GET	`/portfolio`	보유 자산 조회
POST	`/portfolio/rebalance`	순수함수 리밸런싱 제안 + LLM 해석
POST	`/copilot/plan`	자연어 → multi-step 계획
POST	`/copilot/query`	SSE 스트리밍 카드 응답
GET	`/copilot/session/{id}`	멀티턴 세션 조회
DELETE	`/copilot/session/{id}`	세션 삭제

단일 진실 스키마: `shared/openapi.json` → `shared/types/` 에 `openapi-typescript` 로 자동 생성.

8. 프론트엔드 페이지 구조

```
frontend/app/
├── page.tsx                    # 홈: CSV 업로드 + 대시보드 렌더
├── portfolio/page.tsx          # 보유 자산 + 리밸런싱 슬라이더 + 프리셋 3종
├── watchlist/page.tsx          # 추적 목록 CRUD
└── symbol/[market]/page.tsx    # 종목 상세: TradingView 차트 + 분석 패널
```

주요 컴포넌트: `analyzer-result-panel`, `router-reason-panel`, `rebalance-panel`, `command-bar`, `concentration-risk-alert`. 상태관리: **Zustand** (UI) + **TanStack Query** (서버) 조합.

9. 구현 완료 기능 (Sprint 01~06)

Sprint	주제	iter	최종 점수	주요 산출물
01	그린 베이스라인 재확립	3	8.75/10	`/copilot/plan` 신설, Router heuristic 22 골든 샘플 100%, contract 18 pass
02	E2E 핵심 플로우	2	8.75/10	News/Filing RAG 인프라, portfolio context opt-in, `useTypewriter` mount sync 수정
03	완성도 가시자산	1	8.8/10	Sub-agents 3종 (comparison/simulator/news-rag), CopilotCard 6 variant, 스크린샷 5장
04	데모 시연 UX 정합성	4	9.0/10	SSE 스트리밍 `/copilot/query`, Command Bar, `ConcentrationRiskAlert` 타입 가드, contract → pytest 전환
05	기술 깊이 문서 + 세션 메모리	1	10.0/10	ADR 0008 리밸런싱, 세션 메모리 + follow-up 라우팅, SQL 주입 방지, mermaid 3종
06	제출 패키지 최종 점검	1	10.0/10	리허설 로그, checklist 49항목, 수치 일관 갱신

누적: 12 iter · 평균 9.2/10 · 322 pytest + 80 vitest 그린.

10. 품질·성능 지표

항목	실측값	기준
Lighthouse Performance	90~92 (4페이지)	≥ 90
Lighthouse Accessibility	93~96	≥ 90
Mypy (strict)	53 파일 0 이슈	0
Ruff	0 경고	0
tsc	0 에러, 2 warning	0 error
Pytest (BE)	322 passed	100% green
Vitest (FE)	80 passed	100% green
Contract (ASGI 격리)	18 passed	100% green
Router heuristic 정확도	22/22 (100%)	≥ 90%
SSE 첫 카드 p50	< 1.5s	< 2s

11. 개발 프로세스 — 자동 반복 하네스 (ADR 0009)

Planner → (Generator → Evaluator) *루프를 스프린트당 1~4 iter 돌려 수락 기준 통과까지 자동 반복*.

- **Planner** (Opus): 1~4문장 피처 요청 → 스프린트 분해 + 수락 기준 stub

- **Generator** (Sonnet/Opus): 실제 코드 변경, `.claude/agents/*` 중 적절한 역할에 위임
- **Evaluator** (Sonnet): `make` 훅 실행 + 0~10점 채점 + 실패 지점 지적
- **종료 조건**: 점수 ≥ 8.0 or max_iter=5
- **독립 LLM 호출**: Generator와 Evaluator는 별 프로세스 → 자기 평가 편향 차단

산출물: `docs/harness-runs/<timestamp>/` (summary.md, sprint-NN-M.json, role-reports/).

12. 주요 ADR

번호	제목	상태
0001	스택 선정 (Next.js 15 + FastAPI + LangGraph)	Accepted
0007	포트폴리오 컨텍스트 주입 opt-in	Accepted
0008	리밸런싱 제안 = 순수함수 + LLM 해석 분리	Accepted
0009	개발 타임 자동 반복 하네스	Accepted

13. 데모 시나리오 (8분)

1. **[0:00]** CSV 업로드 (코인 + 주식 혼합 샘플)
2. **[0:05]** Router가 heuristic으로 자산군 판별 → `(heuristic)` 마커 표시
3. **[0:10]** Analyzer 병렬 실행 → 카드 progressive 렌더 (SSE)
4. **[1:30]** Copilot Command Bar — "삼성전자 vs TSMC 1년 수익률 비교" 자연어 질의
5. **[2:30]** Comparison 서브에이전트 → `comparison_table` 카드 + citation
6. **[4:00]** 포트폴리오 페이지 → 슬라이더로 공격형/균형형/안정형 프리셋
7. **[5:00]** 리밸런싱 액션 확인 (순수함수 수치 + LLM 해석)
8. **[6:00]** 집중도 경고 (`ConcentrationRiskAlert`) 데모
9. **[7:00]** 3단 게이트 통과 메타 노출 + Router 결정 트리 다이어그램 설명

참조: `docs/qa/demo-rehearsal-2026-04-22.md` .

14. 공모전 심사 정렬

심사 축	달성도	근거
기술적 깊이	95%	Router/Analyzer mermaid 3종, ADR 4건, 3단 게이트 명시적 구현
데모 임팩트	90%	5초 내 첫 카드, SSE progressive 렌더, Command Bar 자연어 인터페이스
완성도	92%	Lighthouse 90+ (4페이지), 다크모드, 로딩/에러/빈 상태
코드 품질	93%	mypy/ruff/tsc 0 경고, 322+80 테스트 그린
종합 예상	**~92%**	하네스 평균 9.2/10

15. 미구현·리스크·향후 작업

15.1 현재 리스크

리스크	영향	완화
실시간 시세 미연동	리밸런싱 시 avg_cost 폴백	ADR 0008 에 명시, v2 범위
FX 리밸런싱 0 고정	환율 비중 조정 불가	데모 샘플에서 FX 제외
공모전 당일 LLM 레이트리밋	데모 실패 위험	프롬프트 캐시 + 데모 샘플 사전 녹화 백업

15.2 v2 로드맵

1. **실시간 시세**: Upbit WebSocket + Yahoo Finance 통합
2. **신규 종목 추천**: 포트폴리오 외부 자산 매수 제안 (risk-return 최적화)
3. **사용자 인증**: NextAuth + Neon Row Level Security
4. **모바일 PWA**: 홈스크린 설치 + 오프라인 캐시
5. **다국어**: 영어·한국어 i18n

16. 금지 사항

- CLAUDE.md / `.claude/rules/*` 단독 수정 금지 (팀 합의 후 PR)
- 실거래 API 키 커밋 금지 (데모용 고정 샘플만)
- LLM 응답을 3단 게이트 없이 UI 직결 금지
- `any` / `# type: ignore` 무단 사용 금지 (PR 사유 명시)

부록 A. 디렉토리 구조

```
hacker-dashboard/
├── frontend/              # Next.js 15 앱
│   ├── app/               # App Router 페이지
│   ├── components/        # UI 컴포넌트
│   └── e2e/               # Playwright 테스트
├── backend/               # FastAPI + LangGraph
│   ├── app/
│   │   ├── agents/        # Router, Analyzers, Gates, graph.py
│   │   ├── api/           # 라우트
│   │   └── services/      # 도메인 서비스 (순수함수)
│   └── tests/contract/    # ASGI 격리 contract 테스트
├── shared/                # openapi.json + types/
├── docs/
│   ├── adr/               # 아키텍처 결정 기록
│   ├── agents/            # Router 결정 트리
```

```
│   ├── harness-runs/      # 하네스 실행 로그
│   └── qa/                # 리허설·checklist
└── .claude/               # 팀 규약·에이전트·스킬
```

부록 B. 팀 (Agent Teams 4인 병렬)

역할	담당
`frontend-engineer`	Next.js 페이지·컴포넌트·차트·상태관리
`backend-engineer`	FastAPI 라우트·LangGraph 노드·DB
`analyzer-designer`	Router/Analyzer 프롬프트·품질 게이트 설계
`integration-qa`	E2E·계약 테스트·배포·데모 리허설

`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` 환경에서 독립 컨텍스트로 병렬 동작.

부록 C. 참고 문서

- 하네스 런 최종 summary: `docs/harness-runs/20260422-103115/summary.md`
- 아키텍처 결정: `docs/adr/0001-stack-selection.md`, `0007-portfolio-context-injection.md`, `0008-rebalance-proposal.md`, `0009-dev-harness.md`
- Router 결정 트리: `docs/agents/router-decisions.md`
- 팀 규약: `.claude/rules/conventions.md`, `CLAUDE.md`

이 PRD는 2026-04-22 기준 브랜치 `feat/qa-harness-20260422-111542-nl-copilot-sprint-05` 의 상태를 반영합니다.